Health Insurance Claims App – Project Overview

A Flask + SQLAlchemy (MySQL; SQLite for tests) web app that simulates health-insurance claim handling. All data is fictional and payouts are **simulated** (no real money). Four roles submit, review, approve and settle claims through a strict, audited workflow.

1. Architecture

Layer	File(s)	Job
Entry	run.py, wsgi.py	Start the app (run.py = dev with auto tables; wsgi.py = *flask --app wsgi run*).
Factory	app/__init__.py	create_app: config, DB, CSRF, blueprints, filters (*inr*, *label*), officer queue badge, 403/404 page.
Config	config.py	DB URL (MySQL default, DATABASE_URL override), SECRET_KEY, secure cookies.
Security	auth.py, extensions.py	load_user (session → g.user), login_required(roles) guard; db + CSRF objects.
Routes	routes_main / routes_claims / routes_admin	Thin pages: read request, call services, render template.
Business rules	services.py	ALL rules: visibility, validation, state machine, payout math, audit log. Nothing trusts the browser.
Data	models.py, schema.sql, make_schema.py	8 tables; schema.sql generated from models.
Seed	seed.py + data/*.csv	*flask init-db* creates tables; *flask seed* loads demo data (one chosen password).
UI	templates/*.html, static/app.js, style.css	Jinja pages; JS only filters members by policy, confirms Reject, previews payout.
Tests	tests/test_app.py	16 tests: access, workflow, payout, scoping, CSV, persistence.

Request flow: Browser → Flask route → **login_required** (role check) → services function (rules) → database → template → HTML. State-changing actions go through **act()**: lock claim row → run rule → commit (or rollback + error message) → redirect back to the claim.

2. Roles

Role	Demo users	Can do	Sees
Policyholder	aasha, bilal, chitra, dev, esha, farhan, gita, harish	Create/edit own **reimbursement** drafts, tick document checklist, submit, answer more-info requests, view own policies.	Only claims on own policies (own drafts only).
Hospital desk	city_, lotus_, green_, sunrise_hospital	Create **cashless** claims for own network hospital only; same draft/checklist/submit/respond steps.	Only own hospital's cashless claims.
Claims officer	officer_meera, officer_rahul	Assign claim to self, start review, request more info, approve (eligible amount) or reject (reason).	All non-draft claims; review queue.
Admin	admin_demo	Record the simulated settlement; manage hospitals, plans, users.	All non-draft claims; admin page.

All demo accounts share the one password entered at seed time (min 8 chars).

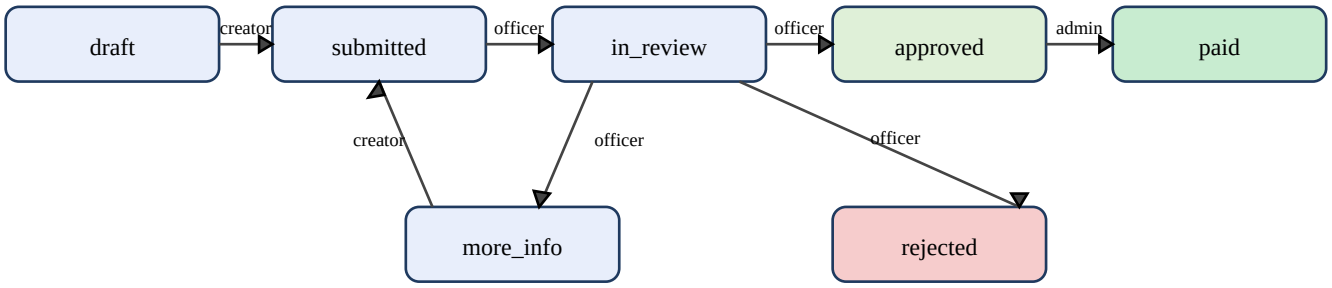
3. Pages (URL → who → what)

Page	URL	Roles	Purpose
Login	/login	anyone	Username/password; same error for either wrong. Logout is POST /logout.
Dashboard	/	all	KPI cards (total, open, to-settle, paid), status tabs, 8 newest claims; officers also see unassigned and my open.
Claim list	/claims	all	Search by id/member name; filter status, type, hospital; CSV export.
CSV export	/claims/export.csv	all	Filtered, visible claims; cells formula-safe.
Review queue	/review	officer	Open claims (submitted, in_review, more_info) with assign buttons.

Page	URL	Roles	Purpose
New claim	/claims/new	holder, hospital	Form; type fixed by role. Save draft or Submit.
Edit claim	/claims/<id>/edit	holder, hospital	Edit own draft only.
Claim detail	/claims/<id>	all (if visible)	Policy snapshot, claim data, document checklist, officer warnings, payout breakdown, timeline, allowed buttons.
Policies	/policies	holder	Own policies, plan benefits, members, remaining cover.
Hospitals	/hospitals	all	Directory with cashless-network flag.
Reports	/reports	all	Counts and requested/eligible/paid totals by status and type (visible claims only).
Settlements	/settlements	all	Totals, approved claims awaiting payout, ledger of settlements.
Admin	/admin	admin	Add/edit hospitals, plans, users (no delete).
Error	(403/404)	all	Friendly error page.

Templates: base.html (layout + role-based sidebar), _macros.html (shared widgets), plus one template per page above.

4. Claim lifecycle



Transition	Who	Action route / service	Rule
draft → submitted	creator	/submit · submit_draft	Strict re-validation (no future admission date).
submitted → in_review	assigned officer	/assign then /start · assign, start_review	Assign is one conditional UPDATE: two officers cannot both win.
in_review → more_info	assigned officer	/more-info · request_info	Pick document + written reason; document marked missing.
more_info → submitted	creator	/respond · respond_more_info	Written response required.
in_review → approved	assigned officer	/approve · approve	Reason; 0 < eligible ≤ requested and ≤ remaining cover.
in_review → rejected	assigned officer	/reject · reject	Reason mandatory; eligible = 0.
approved → paid	admin	/settle · settle	Amount must equal computed payout; one settlement per claim.

Every transition goes through **change_status** (checks allowed move + right person) and writes a ClaimHistory row (sequence no, from/to, actor, note, time) in the same transaction – an audit trail with no partial changes.

5. Step-by-step flows

Reimbursement (policyholder): Login → New claim (policy, member, hospital, treatment, dates, amount, description) → Save draft → tick documents (hospital bill, discharge summary) → Submit → watch status; if officer asks for info, respond → after approval/payment see breakdown.

Cashless (hospital desk): same steps, but hospital is fixed to the desk's own hospital and it must be in the cashless network.

Officer: Review queue → Assign to me → Start review → read warnings (policy inactive on admission, treatment not covered, duplicate claim, missing document, remaining cover) → request info, or approve with eligible amount (live payout preview), or reject with reason.

Admin: Settlements page lists approved unpaid claims → Settle (payee = hospital for cashless, policyholder for reimbursement) → ledger updates. Admin page maintains hospitals, plans and users.

6. Payout formula

Payout = (eligible − **min(eligible, deductible)) × (1** − **co-pay% / 100)**, Decimal only, rounded half-up to 2 places. Example: eligible 20,000, deductible 2,000, co-pay 10% → 16,200.00. Computed by *compute_payout*; the JS preview is display-only and the server recomputes.

7. Visibility and security rules

• **visible_claims** is the single scope rule used by lists, KPIs, reports, CSV, detail pages: holder = own policies; hospital = own cashless claims; staff = non-draft; drafts visible only to their creator.

• Non-existent claim → 404; someone else's claim → 403 (typing URLs does not leak data).

• CSRF tokens on all forms; HttpOnly + SameSite=Lax cookies; password hashes (never plain text); session cleared on login/logout.

• Claim type is derived from role, never from the form; hospital/member/policy checked server-side.

• Row lock (FOR UPDATE) during actions; CSV formula-injection protection; buttons shown are UI hints only – each action is re-checked.

• Duplicates are only a warning for officers, never auto-rejected.

8. Data model (8 tables)

Table	Holds	Key links
hospitals	name, city, network_status (cashless allowed)	—
users	username, password hash, role, display name	hospital_id (desk users)
plans	annual limit, deductible, co-pay %, covered treatments (a|b)	—
policies	dates, status active/expired	holder user, plan
members	covered person, relationship	one policy
claims	type, treatment, dates, requested/eligible amount, status, reason	policy, member, hospital, officer, creator
claim_documents	checklist: received/missing per document type (metadata only)	claim (unique per type)
claim_history	audit event per transition	claim, actor (unique sequence)
settlements	simulated payee, amount, who/when	claim (one per claim)

Seed data: 5 hospitals, 3 plans, 15 users, 8 policies, 11 members, 13 claims, 2 settlements (CSV files in /data).

9. Run it

1) Create MySQL DB *claims_db* (or set DATABASE_URL, e.g. sqlite). 2) *pip install -r requirements.txt*. 3) Set DB_USER/DB_PASSWORD/DB_HOST/DB_NAME and SECRET_KEY as environment variables (prefer env vars over editing config.py; it currently contains a hard-coded default DB password that should be removed). 4) *flask --app wsgi init-db* then *flask --app wsgi seed*. 5) *python run.py* → http://127.0.0.1:5000. Tests: *pytest*.